

Soren

A Reinforcement Learning Agent for Source-Level Vulnerability Localization

Soren is a reinforcement learning system for **source-level vulnerability localization**. Instead of simply predicting whether a function is vulnerable, it learns **how to navigate a program's Control Flow Graph (CFG)** to identify the exact statement responsible for a security vulnerability.

Most existing vulnerability detection models use Graph Neural Networks (GNNs) or Transformer-based architectures that analyse an entire function in a single forward pass and output a binary prediction. While these models achieve strong detection performance, they do not model the search process involved in locating the vulnerable statement.

Soren reformulates this as a **sequential decision-making problem**, making it a natural application of reinforcement learning.

Each function is first converted into a **Control Flow Graph (CFG)**, where nodes represent statements or basic blocks and edges represent possible execution paths. This CFG serves as the environment for the RL agent.

At the beginning of each episode, the agent starts at a node in the graph. Its state consists of the current node along with lightweight structural features such as node type, in/out degree, neighbouring nodes, and whether the node represents a branch, function call, assignment, or return statement. The agent then chooses one of three actions: move to a neighbouring node, backtrack, or declare that the current node is vulnerable.

The agent is trained using a standard reinforcement learning algorithm such as **PPO** or **DQN**. The reward function encourages efficient exploration by assigning a small penalty for every movement, a large positive reward for correctly identifying the vulnerable node, and negative rewards for incorrect localization or excessive traversal. Over repeated episodes, the agent learns a policy that efficiently follows execution paths leading to vulnerable code.

The project will be trained and evaluated on datasets such as **BigVul**, focusing on one or two vulnerability categories (CWEs) to keep the problem manageable. Its performance will be compared against baseline graph traversal strategies such as Random Walk, Breadth-First Search (BFS), and Depth-First Search (DFS), using metrics including localization accuracy, average search steps, success rate, and cumulative reward.

Unlike existing approaches that perform graph classification in a single inference step, Soren explicitly learns a **navigation policy** over the program's structure. This makes the reasoning process interpretable - the agent's traversal through the CFG can be visualized in real time,

allowing users to observe how it explores execution paths before converging on the vulnerable statement.

By treating vulnerability localization as a graph navigation problem rather than a graph classification task, Soren explores a relatively underexplored application of reinforcement learning while demonstrating how RL can learn intelligent search strategies for software security analysis.

Team:

- Karan Kumar (23BPS1034)
- Aryan Kumar (23BPS1193)